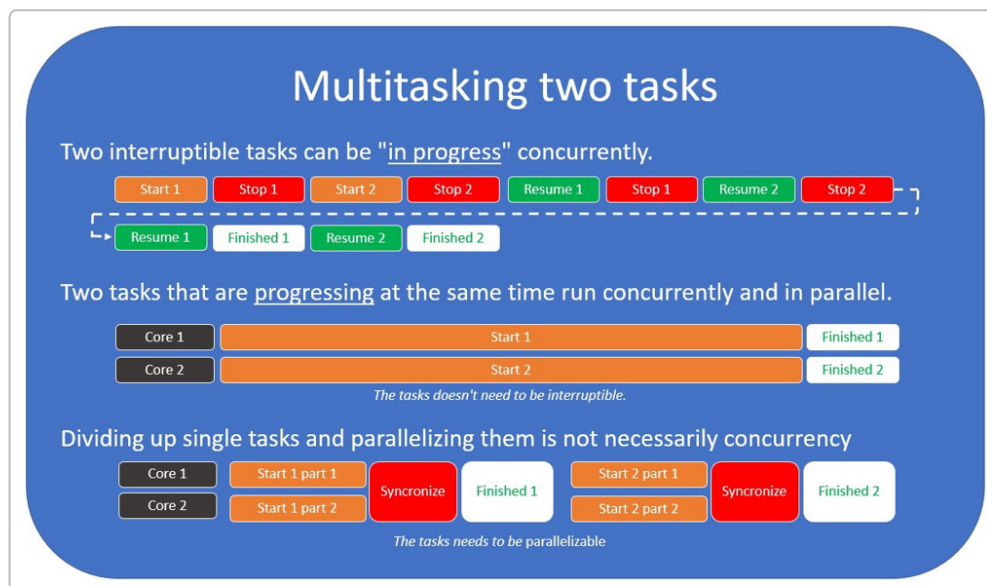


Асинхронность и многопоточность в Rust: системные основы

Конкурентность vs. параллелизм и виды многозадачности

Конкурентностью называют способность программы **иметь несколько задач в работе одновременно**, тогда как **параллелизм** – это выполнение многих задач **буквально одновременно** (например, на разных ядрах) ¹. Иначе говоря, параллельные задачи действительно идут в один момент времени, а конкурентные могут чередоваться на одном потоке выполнения. Конкурентность требует, чтобы задачи были *прерываемыми*: их выполнение можно приостановить и позже возобновить ². Именно прерываемость позволяет единственному потоку *чередовать* несколько задач (многозадачность), тогда как параллелизм задействует дополнительные ресурсы (например, другой ядро CPU) для одновременного прогресса нескольких задач.



Пример: визуализация многозадачности для двух задач. Вверху: две задачи выполняются конкурентно на одном ядре – они прерываются и возобновляются по очереди (кооперативная многозадачность). В середине: две задачи выполняются параллельно на двух ядрах (истинный параллелизм). Внизу: разбиение одной задачи на части для параллельного выполнения (параллелизация без отдельной независимой второй задачи).

Многозадачность в современных системах реализуется преимущественно как *вытесняющая (принудительная) многозадачность*. Ранние системы (например, Windows 3.1) использовали *кооперативную* модель, где задача сама уступала процессор вызовом особого API – сбоя одной программы мог подвесить всю систему ³. В *вытесняющей* модели ответственность за переключение между задачами берет на себя ОС: она *прерывает* выполнение текущего потока по

таймеру или событию и переключается на другой поток, возвращаясь к первому позже ⁴ ⁵ . Благодаря быстрому *планированию* (сотни переключений в секунду) создается иллюзия одновременной работы множества процессов даже на одном ядре, и интерфейс остается отзывчивым ⁶ ⁵ . Современные ОС всегда используют вытесняющую многозадачность, что избавляет разработчиков от ручного уступания CPU, повышая стабильность системы.

Планирование потоков операционной системой

Потоки (threads) – единицы исполнения внутри процессов – планируются *диспетчером* ОС на выполнение на CPU. Диспетчер решает, какой поток когда и на каком ядре исполняться, и может приостанавливать и возобновлять их по своему усмотрению. Это происходит с помощью *аппаратных прерываний* (например, периодического таймера): процессор прерывает текущую задачу и передает управление ядру ОС, которая сохраняет контекст потока и загружает контекст другого ⁷ . **Контекстное переключение** включает сохранение регистров, смену стека и т.п., что позволяет возобновить приостановленный поток позже с того же места. Каждый поток имеет собственный стек, что увеличивает потребление памяти, а создание потока – сравнительно тяжелая операция (включает системный вызов, выделение памяти под стек ~MB, инициализацию) ⁸ ⁹ . Тем не менее, вытесняющее планирование ОС обеспечивает изоляцию: даже если поток завис в бесконечном цикле, ОС рано или поздно отберет у него CPU и переключится на другие задачи.

Важно понимать различные *референсные кадры* времени выполнения программы ¹⁰ . С точки зрения самого потока его код выполняется последовательно, но с точки зрения ОС выполнение может в любой момент прерваться и возобновиться позже, а с точки зрения CPU команды разных потоков вообще перемешаны во времени. Процессор исполняет инструкции по одной, не “зная” о принадлежности к разным потокам, а при аппаратном прерывании мгновенно переключается на обработчик – так аппаратно реализуется конкурентность ¹¹ ¹² . В однопоточном контексте все перестановки инструкций незаметны, но в системном масштабе потоки исполняются вперемежку.

Гиперпоточность и многопроцессорность

Для полноты: современные CPU поддерживают *гиперпоточность* (Simultaneous Multithreading) – технологию, позволяющую одному ядру исполнять два потока одновременно, используя неиспользуемые блоки внутри ядра ¹³ . Так один физический ядро может появляться как два “логических” (например, 4 ядра/8 потоков), увеличивая пропускную способность ~30% за счет аппаратного чередования инструкций разных потоков ¹⁴ . Кроме того, большинство систем теперь *многопроцессорные*: несколько физических ядер могут действительно выполнять код параллельно. Это приводит к тому, что часть задач может идти параллельно, а часть – конкурентно (вытесняюще) на каждом ядре. Правильное использование всех ядер требует *параллельного программирования*, тогда как эффективная загрузка одного ядра без простаивания – задача *конкурентного (асинхронного) программирования*.

Асинхронное программирование и неблокирующий I/O

Асинхронность – способ писать код так, чтобы длительные операции (например, ввод-вывод) не простаивали впустую. Вместо блокирования потока на ожидании, задача *приостанавливается* (выпуск управления) до наступления события, позволяя выполнить другой код в промежутке. В Rust

современная асинхронность построена на *стек-независимых корутинах* (stackless coroutines) – это и есть `async/await`. Ключевая идея: `async fn` при компиляции превращается в *станочный автомат* (тип «Future»), который может приостанавливать свое выполнение на точках `.await` и возвращать управление *исполнителю* (runtime) ¹⁵ ¹⁶. Несколько таких задач (futures) могут запускаться в *одном ОС-потоке* и кооперативно разделять CPU: пока одна ожидает внешнего события, исполнитель переключается на другую задачу. В отличие от потоков ОС, переключение между `async`-задачами не требует системного контекста: достаточно вызова функции (poll) и хранения небольшого состояния задачи в куче, поэтому тысячи задач могут эффективно мультиплексироваться на одном потоке.

Runtime (исполнитель) асинхронных задач обычно реализует *реакторно-исполнительную* модель. **Реактор** – это цикл обработки событий, связанный с неблокирующим I/O. В Unix-подобных ОС для этого применяется системный вызов **epoll** (в Windows аналог – IOCP, в macOS – kqueue). **Epoll** предоставляет *очередь событий*: программист регистрирует интерес к событиям (например, «сокет готов для чтения»), а затем вызывает `epoll_wait`, который *блокирует поток до наступления события* или таймаута ¹⁷ ¹⁸. Блокировка здесь «умная»: поток спит внутри `epoll_wait`, а не занимает CPU. Когда, например, на соquete появились данные, ОС **будит** поток, и `epoll_wait` возвращает информацию о готовых дескрипторах. В *readiness*-модели (epoll, kqueue) событие означает «ресурс готов к операции» – например, сокет готов к чтению (данные есть в буфере) ¹⁹ ²⁰. В *completion*-модели (Windows IOCP) событие означает завершение операции – т.е. *данные уже прочитаны в буфер* ²¹ ²². Разница важна для абстракций: Rust-экосистема обычно использует модель готовности, т.к. она унифицирована между Unix-системами, а под Windows можно эмулировать ее (через обертку поверх IOCP) ²³ ²⁴.

В типичном Rust-исполнителе (например, Tokio) один или несколько потоков ОС занимаются асинхронными задачами. Они совместно используют один **реактор** I/O на основе epoll:

1. Задачи, выполняя сетевые вызовы, регистрируют интерес в реакторе и добровольно приостанавливаются (`.await`).
2. Реактор через `epoll_wait` засыпает, пока не произойдет событие (например, поступление данных).
3. Как только ОС пробуждает его с событием, соответствующая задача помещается в очередь на исполнение.
4. **Исполнитель** (executor) возобновляет задачу, передав ей управление, и она продолжает с места ожидания.

Таким образом достигается *кооперативная многозадачность* без блокирования потоков: ни один поток не простаивает зря – когда одна задача ждет I/O, CPU переключается на другие задачи *на том же потоке*. Системные потоки при этом сами планируются ОС вытесняясь на разных ядрах (для параллелизма). Асинхронность в Rust позволяет эффективно обслуживать множество одновременно ожидающих операций (например, тысячи сетевых соединений) без создания огромного числа потоков ОС.

Почему нужен epoll? Исторически, альтернатива – каждый сокет на отдельном блокирующем потоке или опрос (polling) множества сокетов в цикле. Первый подход не масштабируется (слишком много потоков), второй неэффективен (тратит CPU даже в ожидании). *Epoll* и аналогичные мультиплексоры решают проблему эффективно, особенно на большом количестве дескрипторов ²⁰. Epoll использует внутренние структуры ядра и уведомления: в режиме ожидания поток «спит» в ядре, а пробуждается

только когда есть готовое событие, что экономит CPU. Таким образом, epoll – основа большинства высокопроизводительных сетевых серверов на Linux и лежит под капотом mio /Tokio в Rust.

Атомарные операции и модель памяти

Переходя к *многопоточности* (параллельным потокам исполнения): основная сложность – **синхронизация памяти** между потоками. Процессор и компилятор вправе *переставлять и оптимизировать инструкции* в пределах одного потока, не нарушая логики *однопоточного* исполнения. Однако такие перестановки делают непредсказуемым порядок, в котором один поток «видит» действия другого. В идеале для программиста наиболее интуитивна *последовательная консистентность* (Sequential Consistency) – когда все операции происходят в едином глобальном порядке, соответствующем порядку в исходном коде каждого потока. Но ради производительности реальность слабее: архитектуры и компиляторы допускают выполнение out-of-order, буферизацию записей, и т.д., сохраняя иллюзию последовательности *только внутри одного потока* (в отсутствии спец. мер) ²⁵. Действительно, «перестановки инструкций невидимы внутри однопоточной программы», а обеспечение согласованности между потоками сводится к тому, чтобы *запретить определенные перестановки* там, где это важно ²⁵.

Модель памяти Rust (заимствована из C++11) определяет, какие межпоточные взаимодействия *разрешены*, а какие – нет. *Data race* (гонка данных) – одновременная запись и чтение той же памяти без синхронизации – не определена (Undefined Behavior) и должна исключаться. Для безопасного взаимодействия Rust предоставляет *атомарные типы* (`std::sync::atomic::*`) и *синхронизирующие примитивы* (`Mutex`, `Channel` и пр.). **Атомарной** называется операция, выполняющаяся *неразрывно*, без возможности наблюдать ее промежуточное состояние. Атомики гарантируют, что параллельные обновления переменной не «сломают» данные (например, 64-битное значение не прочитается наполовину обновленным). Но одной атомарности мало – нужна определенность порядка видимости этих операций между потоками. Для этого каждый атомик в Rust имеет параметр типа `Ordering` – требование по *модели памяти*. Основные варианты:

- **Relaxed** – свободная (минимальная) синхронизация. Гарантируется атомарность (нет разрыва данных), но **не** гарантируется упорядочение относительно других операций. Используется, когда нас интересует только *количество/факт* операций, а не порядок.
- **Release** – *освобождающая* запись (store) или модификация. Гарантирует, что *все операции до нее* в этом потоке станут видимы другим потокам, прочитавшим этот атомик с соответствующей Acquire-семантикой. Иначе говоря, **запись с Release не “пропустит” предыдущие записи* – они случатся до нее по межпоточному порядку ²⁶ ²⁷.
- **Acquire** – *захватывающая* операция чтения (load) или модификация. Гарантирует, что все *последующие* операции в текущем потоке произойдут *после* этой по памяти, то есть не могут быть переупорядочены перед ней ²⁶ ²⁷. Обычно Acquire-чтение пары к Release-записи того же атомака позволяет «увидеть» данные, записанные до Release.
- **AcqRel** – комбинированная семантика для операций «чтение-модификация-запись» (RMW), одновременно выполняет требования Acquire и Release. Например, `fetch_add` с AcqRel при чтении значения действует как Acquire, при записи нового – как Release ²⁸ ²⁹.
- **SeqCst** – *последовательная консистентность*. Самый строгий режим, дополнительно устанавливающий *глобальный порядок* между всеми SeqCst-операциями на всех атомаках. Любые два SeqCst-вызова по всей программе увидят один и тот же порядок друг относительно друга ³⁰ ³¹. Это упрощает рассуждения, но потенциально самые дорогие операции; на

некоторых архитектурах SeqCst требует дополнительной синхронизации (барьеров). Обычно SeqCst по умолчанию избыточна – можно обойтись сочетанием Acquire/Release для конкретных зависимостей, и в новых кодах часто рекомендуют их.

Rust, следуя модели, запрещает так называемое *наблюдаемое установление из будущего* (out-of-thin-air values) и другие парадоксы, но требует от разработчика правильно выбирать уровни Ordering. Стоит отметить, что на архитектуре x86_64 большинство операций и так выполняются упорядоченно: процессор гарантирует *прямой порядок* памяти (не переупорядочивает обычные записи с обычными чтениями и наоборот) ³² ³³. Поэтому на x86 семантики Acquire/Release зачастую «бесплатны» – не требуют дополнительных инструкций ³⁴ ³⁵ (например, обычная атомарная запись на x86 уже имеет поведение Release). На ARM же память упорядочена слабее: там для обеспечения тех же гарантий требуются специальные инструкции, и потому на ARM атомики с Release/Acquire обходятся дороже, практически сопоставимо с SeqCst ³⁶ ³⁷. Разработчику важно помнить об этих различиях, хотя Rust абстрагирует их: код просто указывает требуемый Ordering, а компилятор сам вставит нужные барьеры для конкретной архитектуры.

Барьеры памяти (Memory Fences)

Помимо атомарных операций с встроенными гарантиями, модель памяти допускает и явные барьеры (memory fences). В Rust это функция `std::sync::atomic::fence(Ordering)`, которая вводит *заградительный барьер* по памяти, не связанный с конкретной переменной. Барьер определенного типа (*Acquire*, *Release*, *AcqRel* или *SeqCst*) запрещает переупорядочивание вокруг себя соответствующих операций. Например, **Release-барьер** не позволит никаким записям **после** него сдвинуться до него, а **Acquire-барьер** не пустит вверх чтения/записи, которые идут **до** него ²⁶ ²⁷. **Полный SeqCst-барьер** останавливает любую перестановку операций через себя (ни до него, ни после него не переедет) ²⁶ ²⁷. Барьеры используются довольно редко – обычно проще привязать упорядочение к конкретным атомикам, – но бывают нужны в нестандартных ситуациях (например, для упорядочения диапазона операций или синхронизации с внешним кодом/оборудованием).

На практике барьер транслируется в специальную инструкцию архитектуры. Как упоминалось, на x86_64 для Acquire/Release барьеров отдельные инструкции не нужны (достаточно обычных правил порядка), поэтому `fence(Acquire)` или `fence(Release)` на x86 компилируются в пустую строку – они ничего не делают на этой архитектуре ³⁴ ³⁵. Только полный `fence(SeqCst)` порождает команду `MFENCE` на x86 – она останавливает выполнение до завершения всех предыдущих операций памяти ³⁴. На ARM же барьеры необходимы: там `dmb ish` (Data Memory Barrier) с различными доменами/режимами выступает в роли и полного, и частичного барьера ³⁸ ³⁹. Таким образом, Rust позволяет явно укрепить порядок, если автоматических гарантий атомиков недостаточно. Впрочем, неправильное использование барьеров чревато потерей производительности или даже ошибками, поэтому их применяют осмотрительно. В обычном же коде более распространены *синхронизационные примитивы* верхнего уровня – они внутри себя уже ставят нужные барьеры.

Синхронизация потоков и примитивы ОС (Locks, futex)

Когда нескольким потокам нужен **совместный доступ** к данным, применяют *блокировки* (locks) или другие синхронизационные протоколы. Рассмотрим классический **Mutex** (взаимоисключающая блокировка). Его задача – гарантировать, что в критической секции находится только один поток,

остальные при попытке войти – ожидают. Самый простой способ ожидания – **busy-waiting** (ожидание с прокручиванием): поток в цикле проверяет флаг блокировки, не уступая ядро. Такой *spinlock* легко реализуется с атомиком, и на практике реально используется, если ожидается очень короткое ожидание (несколько наносекунд). Однако, если секция заняла больше времени, впустую крутящийся поток зря тратит процессорное время. Здесь на помощь приходит **блокирующее ожидание**: можно *усыпить поток* до тех пор, пока ресурс не освободится, чтобы CPU занимался другими задачами ⁴⁰.
⁴¹ . Как мы обсуждали, ОС умеет приостанавливать потоки и переключать их. Но нужен механизм, чтобы *разработчик мог явно усыпить поток до какого-то события* – в нашем случае, до освобождения блокировки.

На уровне ОС такой механизм предоставляют системные примитивы синхронизации. В Linux ключевую роль играет системный вызов **futex** (*fast userspace mutex* – «быстрый mutex в пространстве пользователя»). Futex оперирует указателем на 32-битное целочисленное значение в памяти, на которое согласованы несколько потоков ⁴² . Он предоставляет две главных операции: **FUTEX_WAIT** – усыпить текущий поток, *если* по указанному адресу хранится заданное значение, и **FUTEX_WAKE** – разбудить один или несколько потоков, ожидающих на этом адресе ⁴² . Как это применяется для Mutex? Допустим, атомарный флаг **locked** = 0/1 указывает состояние блокировки. Поток-А, выполняя **lock()** для Mutex:

1. Читает флаг. Если 0 (свободно) – пытается атомарно поставить 1 (занять). Если удалось – входит в секцию. Если флаг был 1 (уже занят) – переходит к шагу 2.
2. Выполняет системный вызов **futex_wait(&locked, 1)**. **Ядро** проверяет: если ***locked** всё ещё == 1, поток засыпает (ОС убирает его из планирования). Если же к моменту вызова значение успело стать 0 (например, мьютекс освободили *до того*, как поток успел уснуть), то **futex_wait** немедленно возвращается и *не* усыпляет поток ⁴³ ⁴⁴ . Это важная гарантия: предотвращается «пропуск пробуждения» – если разблокировка случилась одновременно с попыткой заснуть, поток не заснет навечно.
3. Когда другой поток-Б вызовет **unlock()**, он атомарно поставит **locked = 0** и вызовет **futex_wake(&locked, 1)** – тем самым ядро разбудит **один** из ожидающих потоков (если таковые есть) ⁴⁵ ⁴⁶ . Поток-А просыпается и повторяет попытку занять mutex (переходит к шагу 1).

Таким образом реализовано эффективное ожидание: пока блокировка свободна, потоки работают *чисто в user-space* (только атомики); как только возникает конкуренция, лишние потоки уходят в спящее состояние в ядре. **Futex** обеспечил «стык» между пользовательским кодом и планировщиком ОС: поток спит до события, не тратя квоты CPU, и просыпается по сигналу. Это существенно масштабируется – тысячи спящих потоков почти не потребляют ресурсов. Кроме того, futex элегантно решает проблему ложных пробуждений: **futex_wait** всегда проверяет условие «значение всё ещё равно ожиданию» *после* пробуждения, поэтому даже если проснулся по ошибке, он просто снова ляжет спать, не захватив mutex, пока не получит настоящее освобождение ⁴⁷ .

Стоит отметить, что многие ОС теперь имеют аналогичные примитивы: Windows с версии 8 предоставляет **WaitOnAddress** (по сути futex-эквивалент) ⁴⁸ , а стандарты C++20 включили **std::atomic_wait/notify**, чтобы разработчики могли пользоваться такими возможностями прямо из высокоуровневого кода ⁴⁹ . В Rust стандартная реализация **Std::sync::Mutex** под капотом использует системные примитивы (на Linux – именно futex через платформенно-независимую библиотеку *parking_lot*). Точно так же **Condvar** (условная переменная) внутри

вызывает `futex_wait` / `wake` для ожидания сигнала. Таким образом, низкоуровневый механизм `futex` становится строительным блоком для *различных примитивов синхронизации*: `mutex`'ов, семафоров, условных переменных и прочего.

Итоги и практические замечания

В Rust многопоточное программирование опирается на строгую типовую систему: типы и заимствования гарантируют отсутствие *небезопасных гонок данных*. Нельзя поделить изменяемую переменную между потоками без явного синхронизирующего типа (`Arc<Mutex<T>>` или атомика), что избавляет от большого класса ошибок. Асинхронное программирование позволяет писать высокопроизводительные сетевые и ввода-вывода коды, не занимая поток на каждое соединение. Однако, асинхронность *не упрощает* модель памяти: когда задачи всё же исполняются параллельно (например, `async`-runtime может иметь несколько потоков-исполнителей, или классическая многопоточная программа), нужно учитывать описанные принципы. *Память между потоками должна быть согласована*, либо с помощью атомарных операций с правильными барьерами, либо – что гораздо проще и надёжнее – с использованием проверенных примитивов синхронизации (`Mutex`, `RwLock`, `Channels` и т.д.), которые инкапсулируют всю низкоуровневую механику. Понимание же механизмов ОС – планировщиков, системных вызовов (таких как `epoll`, `futex`), и работы процессоров – помогает писать более эффективный и правильный код на Rust, а также разбираться в поведении программ под нагрузкой.

Источники: Использованы материалы книг “Asynchronous Programming in Rust” (Carl F. Samson, 2024) и “Rust Atomics and Locks: Low-Level Concurrency in Practice” (Mara Bos, 2023), включая описания принципов вытесняющей многозадачности ⁴ ⁵, различий конкурентности и параллелизма ¹, работы `epoll` и системных очередей событий ²⁰ ²², а также низкоуровневой синхронизации посредством атомарных операций, барьеров памяти и `futex` ²⁶ ⁴². Эти источники рекомендуется изучить для более детального погружения в тему.

¹ ² ³ ⁴ ⁵ ⁶ ⁷ ⁸ ⁹ ¹¹ ¹² ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁷ ¹⁸ ¹⁹ ²⁰ ²¹ ²² ²³ ²⁴ Samson Carl Fredrik - Asynchronous Programming in Rust - 2024.pdf
file:///file-RR5pYwz1cbw6xVJTzUr7z

¹⁰ Asynchronous Programming in Rust | Programming | Paperback
<https://www.packtpub.com/en-us/product/asynchronous-programming-in-rust-9781805128137>

²⁵ ²⁶ ²⁷ ²⁸ ²⁹ ³⁰ ³¹ ³² ³³ ³⁴ ³⁵ ³⁶ ³⁷ ³⁸ ³⁹ ⁴⁰ ⁴¹ ⁴² ⁴³ ⁴⁴ ⁴⁵ ⁴⁶ ⁴⁷ ⁴⁸ ⁴⁹ Mara Bos - Rust Atomics and Locks Low-Level Concurrency in Practice - 2023.pdf
file:///file-VjANodP6kdjqznLrMsHEED